ASSIGNMENT 3 – TASK 2

User-Defined Exceptions and Propagation in an Online Examination Portal

Course: Programming in Java | CO3 | 7 Marks | Bloom's Level: K4 – Analyse

1. Problem understanding

The portal must reject submissions when the exam window has closed, when the student is not registered for the course, or when the answer file exceeds the storage limit. These rules are checked across the controller, service and storage layers. The design uses meaningful application exceptions so low-level storage details do not leak directly into the user interface.

2. Exception hierarchy

```
Throwable
■■■ Exception
    ■■■ ExamException (checked)
        ■■■ ExamWindowClosedException
        ■■■ CourseNotRegisteredException
        ■■■ SubmissionStorageException
              ■■■ cause: IOException
```

All three portal-specific exceptions extend ExamException, which extends Exception. They are checked exceptions: the service/controller must deliberately handle or declare these expected business-operation failures. The underlying IOException is retained as the cause of SubmissionStorageException.

3. Layered design and propagation

```
Controller
   | calls submit(...)
   v
ExamService
   | validates exam window and course registration
   | calls save(...)
   v
SubmissionStorage
   | catches IOException
   | throws SubmissionStorageException(message, cause)
   v
ExamService -> Controller
Controller catches the specific exception and displays a safe message.
```

The controller is the presentation boundary and handles the propagated exception. The student sees a clear message such as "Submission failed: Could not store answer-file.pdf", while diagnostic code can inspect the original cause.

4. throw, throws and exception chaining

The service method declares `throws ExamException` because it may propagate any portal exception. Validation failures are created using `throw new ExamWindowClosedException(...)` or `throw new CourseNotRegisteredException(...)`. The storage method declares `throws SubmissionStorageException`. When an IOException occurs, it is wrapped with `new SubmissionStorageException("Could not store " + fileName, e)`. Passing `e` preserves the original exception as the cause; `getCause()` returns it.

5. Complete the Code – TODO 1

```
throw new SubmissionStorageException(
    "Could not store " + fileName, e);
```

This produces the assignment's expected output: Saved answer01.pdf; Error: Could not store answer02.pdf; Cause: Disk quota exceeded.

If the IOException is not passed to the constructor, the cause is not retained. `e.getCause()` in the demonstration returns null; therefore calling `e.getCause().getMessage()` causes a NullPointerException. If the program safely prints only `e.getCause()`, the value is `null`.

6. Test scenarios

Scenario	Exception / path	Expected observation
Valid submission	No exception	Saved and accepted
Exam closed	ExamWindowClosedException	Exam-window rejection
Not registered	CourseNotRegisteredException	Registration rejection
File > 500 KB	IOException chained in SubmissionStorageException	Meaningful storage error and original cause

7. Why use a meaningful SubmissionStorageException?

A storage-layer IOException describes a technical detail. SubmissionStorageException gives the upper layers a domain-specific failure they can handle consistently, while preserving the technical cause for debugging. It avoids coupling the user interface to file-system implementation details and supports a clearer user message.

8. Alternative: unchecked custom exceptions

If ExamException extended RuntimeException instead of Exception, it would become unchecked. Methods would not be required to declare it in a throws clause, and callers would not be forced by the compiler to catch or declare it. Explicit handling could still be added. This is less suitable for expected submission outcomes that callers should consciously handle, but may suit programming errors or violated internal invariants.

9. Time and space complexity

For a single submission, validation and wrapping are constant-time operations, O(1), excluding the actual file I/O. Reading/writing a file of b bytes is O(b) time. The exception object and cause reference require O(1) object space, while the captured stack trace uses O(d) space for stack depth d. Chaining retains the cause object and its trace, so the retained diagnostic memory grows with the number and depth of chained exceptions. Returning an error code avoids stack-trace capture and is generally cheaper, but loses typed failure information and requires callers to interpret codes.

10. Conclusion

The layered design uses checked, meaningful exceptions to represent portal failures, propagates them through declared method contracts, and preserves the original IOException through exception chaining. The controller handles the exception and presents a user-friendly message without losing debugging information.

Execution evidence and screenshots should be added from the student's own Java environment before submission.